متدهای لیست (Python Lists) در زبان برنامه‌نویسی پایتون :

در پایتون، listیکی از مهم‌ترین ساختارهای داده است و متدهای زیادی برای اضافه‌کردن، حذف‌کردن، جست‌وجو، مرتب‌سازی و کپی‌کردن عناصر دارد

لیست مجموعه‌ای از چند مقدار است که:

- ترتیب عناصر را حفظ می‌کند
- قابل تغییر است .(mutable)
- می‌تواند عناصر تکراری داشته باشد
- می‌تواند انواع مختلف داده را در خود نگه دارد

مثلاً:

```
numbers = [10, 20, 30, 40]
```

یا:

```
data = [10, "Ali", 3.14, True]
```

متدهای اصلی List در پایتون

لیست در پایتون 11متد اصلی دارد:

متد	کاربرد
append()	اضافه کردن یک عنصر به انتهای لیست
extend()	اضافه کردن چند عنصر به انتهای لیست
insert()	اضافه کردن عنصر در موقعیت مشخص
remove()	حذف یک عنصر بر اساس مقدار
pop()	حذف و برگرداندن عنصر
clear()	پاک کردن تمام عناصر
index()	پیدا کردن موقعیت یک عنصر
count()	شمردن تعداد تکرار یک عنصر
sort()	مرتب کردن لیست
reverse()	معکوس کردن ترتیب لیست
copy()	ایجاد یک کپی از لیست

حالا تک‌تک آنها را بررسی کنیم

متد append() :

کاربرد : برای اضافه کردن یک عنصر به انتهای لیست استفاده می‌شود

Syntax

list.append(element)

مثال

```
numbers = [10, 20, 30]
numbers.append(40)
print(numbers)
```

خروجی:

```
[10, 20, 30, 40]
```

عنصر 40به انتهای لیست اضافه شد

مثال با رشته

```
names = ["Ali", "Sara"]
names.append("Reza")
print(names)
```

خروجی:

```
['Ali', 'Sara', 'Reza']
```

نکته مهم : append()فقط یک عنصر اضافه می‌کند

مثلاً:

```
numbers = [1, 2, 3]
numbers.append([4, 5])
print(numbers)
```

خروجی:

```
[1, 2, 3, [4, 5]]
```

یعنی [4, 5]به عنوان یک عنصر اضافه شده است

متد extend() :

کاربرد : برای اضافه کردن چند عنصر به انتهای لیست استفاده می‌شود

Syntax

list.extend(iterable)

مثلاً:

```
numbers = [1, 2, 3]
numbers.extend([4, 5, 6])
print(numbers)
```

خروجی:

```
[1, 2, 3, 4, 5, 6]
```

تفاوت append() و extend() :

این تفاوت بسیار مهم است.

با append():

```
numbers = [1, 2, 3]
numbers.append([4, 5])
print(numbers)
```

نتیجه:

```
[1, 2, 3, [4, 5]]
```

اما با extend():

```
numbers = [1, 2, 3]
numbers.extend([4, 5])
```

```
print(numbers)
```

نتیجه:

```
[1, 2, 3, 4, 5]
```

یک مثال دیگر

```
letters = ["a", "b"]
letters.extend("cd")
print(letters)
```

خروجی:

```
['a', 'b', 'c', 'd']
```

چون رشته قابل پیمایش (iterable) است، کاراکترهای آن یکی‌یکی اضافه می‌شوند

insert() :

کاربرد : برای اضافه کردن یک عنصر در موقعیت مشخص استفاده می‌شود

Syntax

```
list.insert(index, element)
```

مثال:

```
names = ["Ali", "Reza", "Sara"]
names.insert(1, "Mina")
print(names)
```

خروجی:

```
['Ali', 'Mina', 'Reza', 'Sara']
```

در اینجا "Mina"در index شماره 1قرار گرفت

یعنی:



نکته : عنصر قبلی حذف نمی‌شود؛ عناصر بعدی یک خانه به جلو می‌روند

مثلاً:

```
numbers = [10, 20, 30]
numbers.insert(0, 5)
print(numbers)
```

خروجی:

```
[5, 10, 20, 30]
```

متد remove() :

کاربرد : برای حذف یک عنصر بر اساس مقدار آن استفاده می‌شود

Syntax

```
list.remove(value)
```

مثال:

```
numbers = [10, 20, 30, 40]
numbers.remove(30)
print(numbers)
```

خروجی:

```
[10, 20, 40]
```

اگر عنصر چند بار تکرار شده باشد؟

()removeفقط اولین مورد را حذف می‌کند

```
numbers = [10, 20, 10, 30, 10]
numbers.remove(10)
print(numbers)
```

خروجی:

```
[20, 10, 30, 10]
```

فقط اولین 10حذف شد

اگر عنصر وجود نداشته باشد؟

مثلاً:

```
numbers = [1, 2, 3]
numbers.remove(10)
```

با خطای:

```
ValueError
```

مواجه می‌شویم

متد ()pop :

()popبرای حذف عنصر استفاده می‌شود، اما یک تفاوت مهم با ()removeدارد:

()popعنصر حذف‌شده را برمی‌گرداند و بدون آرگومان است

اگر چیزی به ()popندهیم، آخرین عنصر حذف می‌شود

```
numbers = [10, 20, 30, 40]
x = numbers.pop()
print(x)
print(numbers)
```

خروجی:

```
40
[10, 20, 30]
```

باindex می‌توانیم مشخص کنیم کدام عنصر حذف شود:

```
numbers = [10, 20, 30, 40]
x = numbers.pop(1)
print(x)
print(numbers)
```

خروجی:

```
20
[10, 30, 40]
```

چون 20در index شماره 1قرار داشت

تفاوت remove()و pop() :

numbers.remove(20)

می‌گوییم مقدار ۲۰ را حذف کن

اما:

numbers.pop(1)

می‌گوییم:

عنصری که در index شماره ۱ است را حذف کن و آن را به من برگردان

متد clear() :

کاربرد : برای حذف تمام عناصر لیست استفاده می‌شود

```
numbers = [10, 20, 30, 40]
numbers.clear()
print(numbers)
```

خروجی:

```
[]
```

لیست همچنان وجود دارد، اما خالی شده است

تفاوت با del

مثلاً:

```
numbers = [1, 2, 3]
del numbers
```

در این حالت خود متغیر numbers حذف می‌شود

اما:

```
numbers = [1, 2, 3]
numbers.clear()
```

متغیر همچنان وجود دارد و مقدارش یک لیست خالی است

```
[]
```

متد index() :

کاربرد : برای پیدا کردن indexاولین وقوع یک مقدار استفاده می‌شود

```
names = ["Ali", "Sara", "Reza"]
x = names.index("Sara")
print(x)
```

خروجی:

```
1
```

چون Saraدر index شماره 1قرار دارد

اگر مقدار چند بار وجود داشته باشد

```
numbers = [10, 20, 30, 20, 40]
print(numbers.index(20))
```

خروجی:

```
1
```

چون اولین 20در index شماره 1قرار دارد

مشخص کردن محدوده جست‌وجو

می‌توانیم مشخص کنیم جست‌وجو از کجا شروع شود:

```
numbers = [10, 20, 30, 20, 40]
print(numbers.index(20, 2))
```

خروجی:

```
3
```

یعنی از index شماره 2به بعد دنبال 20می‌گردد

متد count() :

کاربرد : برای شمارش تعداد دفعاتی که یک مقدار در لیست تکرار شده است

```
numbers = [10, 20, 10, 30, 10, 40]

print(numbers.count(10))
```

خروجی:

3

چون 10سه بار در لیست وجود دارد

مثال با رشته

```
names = ["Ali", "Sara", "Ali", "Reza", "Ali"]

print(names.count("Ali"))
```

خروجی:

3

متد sort() :

یکی از مهم‌ترین متدهای لیست است

کاربرد : برای مرتب کردن عناصر لیست استفاده می‌شود

مثلاً:

```
numbers = [40, 10, 30, 20]
numbers.sort()
print(numbers)
```

خروجی:

```
[10, 20, 30, 40]
```

به صورت پیش‌فرض از کوچک به بزرگ مرتب می‌کند

مرتب‌سازی نزولی

با reverse=True:

```
numbers = [40, 10, 30, 20]
numbers.sort(reverse=True)
print(numbers)
```

خروجی:

```
[40, 30, 20, 10]
```

مرتب کردن رشته‌ها

```
names = ["Reza", "Ali", "Sara", "Mina"]
names.sort()
print(names)
```

خروجی:

```
['Ali', 'Mina', 'Reza', 'Sara']
```

نکته بسیار مهم : sort()خود لیست را تغییر می‌دهد و معمولاً مقدار مرتب‌شده را برنمی‌گرداند.

یعنی:

```
result = numbers.sort()
```

مقدار resultبرابر Noneخواهد بود

متد reverse()

کاربرد : ترتیب عناصر لیست را برعکس می‌کند

```
numbers = [1, 2, 3, 4, 5]
numbers.reverse()
print(numbers)
```

خروجی:

```
[5, 4, 3, 2, 1]
```

مثال با رشته

```
names = ["Ali", "Sara", "Reza"]
names.reverse()
print(names)
```

خروجی:

```
['Reza', 'Sara', 'Ali']
```

تفاوت reverse() و sort(reverse=True)

این دو را اشتباه نگیرید

reverse() فقط ترتیب فعلی را برعکس می‌کند:

[3, 1, 5, 2]

تبدیل می‌شود به:

[2, 5, 1, 3]

اما:

sort(reverse=True)

اول عناصر را مرتب می‌کند و سپس به صورت نزولی قرار می‌دهد:

[5, 3, 2, 1]

متد copy() :

کاربرد : برای ایجاد یک کپی از لیست استفاده می‌شود

```
numbers = [10, 20, 30]
new_numbers = numbers.copy()
print(new_numbers)
```

خروجی:

```
[10, 20, 30]
```

اما چرا copy()مهم است؟

فرض کنید بنویسیم:

```
numbers = [10, 20, 30]
new_numbers = numbers
new_numbers.append(40)
print(numbers)
```

خروجی:

```
[10, 20, 30, 40]
```

چرا؟

چون new_numbersو numbersبه همان لیست اشاره می‌کنند

اما با :()copy

```
numbers = [10, 20, 30]
new_numbers = numbers.copy()
new_numbers.append(40)
```

```
print(numbers)
print(new_numbers)
```

خروجی:

```
[10, 20, 30]
[10, 20, 30, 40]
```

در این حالت دو لیست مستقل داریم

جمع‌بندی تمام متدها

یک مثال کامل:

```
numbers = [30, 10, 20]
# اضافه کردن یک عنصر
numbers.append(40)
# اضافه کردن چند عنصر
numbers.extend([50, 60])
# اضافه کردن در موقعیت مشخص
numbers.insert(0, 5)
# حذف بر اساس مقدار
numbers.remove(20)
# حذف بر اساس index
numbers.pop(2)
# پیدا کردن index
print(numbers.index(30))
```

```
# شمارش
print(numbers.count(10))
# مرتب‌سازی
numbers.sort()
# معکوس کردن
numbers.reverse()
# کپی گرفتن
new_numbers = numbers.copy()
# پاک کردن
numbers.clear()
```

یک نکته خیلی مهم: هنگام کار با لیست، توابع و قابلیت‌های دیگری هم زیاد استفاده می‌شوند که متد list نیستند؛ مثلاً:

len()

max()

min()

sum()

sorted()

مثلاً:

```
numbers = [10, 20, 30, 40]
print(len(numbers))
print(max(numbers))
print(min(numbers))
print(sum(numbers))
```

خروجی:

```
4
40
10
100
```

تفاوت مهم این است که:

```
numbers.append(50)
```

append()یک methodمربوط به listاست

اما:

```
len(numbers)
```

len()یک تابع داخلی (built-in function) پایتون است، نه متد لیست